Advanced Algorithms

Lecture Notes

Luca Simonetti

2025–2026

Contents

Chapter 1

String Matching

1.1 Introduction & Exact Pattern Matching

1.1.1 Definition

The exact pattern matching problem is defined as follows:

Problem 1.1.1 (Exact pattern matching)**.** Given an alphabet Σ, and two strings T, $P \in \Sigma^*$, for which $|T| = n$ and $|P| = m$, find all occurrences of P in T, i.e. compute $\{\, i \mid T[i,\, i+m-1] = P \,\}$.

Notation

Throughout these notes, Σ denotes a finite alphabet of constant size ($|\Sigma| \in \mathcal{O}(1)$). For a string S of length $|S|$:

- $S[i]$ denotes the i-th character of S (1-based indexing).
- $S[i, j]$ denotes the substring $S[i]\, S[i+1] \cdots S[j]$ (empty string if $i > j$).
- $S[,j] = S[1, j]$ is the prefix of length j; $S[i,] = S[i, |S|]$ is the suffix starting at i.

A *proper* prefix (resp. suffix) of S is any prefix (resp. suffix) strictly shorter than S itself.

Example 1.1.2. Let $\Sigma = \{a, b\}$, $T = \texttt{ababbab}$ and $P = \texttt{abb}$. The only occurrence of P in T is at position 3.

1.1.2 Naive approach

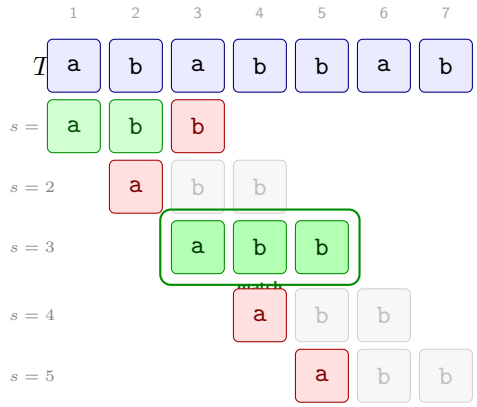


Figure 1.1: Naive exact pattern matching of $P =$ `abb` in $T =$ `ababbab` (1-indexed). Each row corresponds to one shift s; the pattern P is aligned starting at $T[s]$. **Green** = character matched, **red** = first mismatch (algorithm skips to next shift), gray = not compared.

What the Figure 1.1 illustrates is one basic naive approach. In this approach we nest two loops: the outer loop iterates over all possible "first positions" s in T where the pattern P could potentially match, and the inner loop checks character by character if P matches T starting from position s. For example the first row of the figure corresponds to starting the search from index $s = 1$ of T. Given s we then loop one by one each character of P, moving along T as well, and check if all the characters match. In this case we find a mismatch at the third character of P (which is `b`, whereas the corresponding character in T is `a`). When this happens, we break this inner loop and move to the next shift $s = 2$. If the length of the pattern was m and the length of the text was n, the outer loop could have up to $\mathcal{O}(n)$ iterations, and each inner iteration could take up to $\mathcal{O}(m)$ time, leading to a worst-case time complexity of $\mathcal{O}(nm)$ for this naive approach.

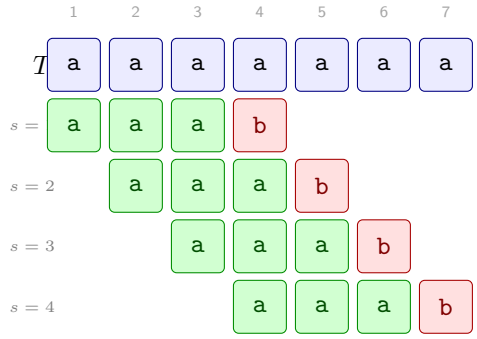


Figure 1.2: Worst-case input for the naive algorithm: $T =$ `aaaaaaa` ($n = 7$) and $P =$ `aaab` ($m = 4$). Every shift s performs $m-1$ successful comparisons before the inevitable mismatch on $P[m] =$ `b`, giving $(n - m + 1)(m - 1) = \Theta(nm)$ total comparisons.

In figure 1.2 we see an example of a worst-case input for the naive algorithm. In this case the text T consists of n `a` characters, and the pattern P consists of $m-1$ `a` characters followed by a `b`. There are $n-m+1$ possible shifts s where

the pattern could potentially match, and for each of these shifts the algorithm will successfully match the first $m-1$ characters of P. In the example of the figure we have $n=7$ and $m=4$, so there are $7-4+1=4$ shifts to check, and for each shift we make m checks, of which $m-1=3$ are successful matches before we encounter the mismatch at the last character of P. This results in a total of $(n-m+1)(m-1)$ comparisons, which is $\Theta(nm)$ in the worst case.

Algorithm 1: Naive exact pattern matching

Input: Text $T[1\ldots n]$, Pattern $P[1\ldots m]$
Output: All positions s such that $T[s, s+m-1] = P$

```
for s ← 1 to n − m + 1 do
    j ← 1
    while j ≤ m and T[s + j − 1] = P[j] do
        j ← j + 1
    if j > m then
        output s
```

Correctness

To pin down what "the inner loop does" precisely, let's give a name to the obvious quantity: for any position i in T, write

$$\text{common}(i) \;=\; \max\{\, k \geq 0 \mid T[i, i+k-1] = P[1,k] \,\}$$

for the length of the longest common prefix of $T[i,]$ and P — basically, how many characters of P would match if you aligned it at position i.

Lemma 1.1.3. *At shift s, the inner loop exits with $j = \text{common}(s) + 1$.*

Proof. Track the invariant: entering each iteration, we have $T[s, s+j-2] = P[1, j-1]$ (the first $j-1$ characters already matched). The loop keeps going as long as the next character also matches; it stops the moment it doesn't, or when j runs past m. Since $\text{common}(s)$ is exactly how many consecutive characters agree, the loop stops right at $j = \text{common}(s)+1$. □

Theorem 1.1.4. *The naive algorithm finds every occurrence of P in T, and nothing more.*

Proof. The outer loop visits every shift $s \in \{1, \ldots, n-m+1\}$, so no candidate starting position is ever skipped. By Lemma 1.1.3, after the inner loop we have $j = \text{common}(s)+1$; the algorithm outputs s exactly when $j > m$, i.e. $\text{common}(s) \geq m$, i.e. $T[s, s+m-1] = P$ — which is precisely the definition of an occurrence. □

Lower bound

Theorem 1.1.5. *Any algorithm for exact pattern matching must make $\Omega(n+m)$ character comparisons in the worst case.*

Proof. Two independent arguments, one for each term.

- **You have to read all of** P ($\Omega(m)$). Suppose the algorithm skips position $P[q]$. An adversary flips just that character, producing a different pattern P'. The algorithm sees the same comparisons on both inputs and therefore gives the same output — which can't be correct for both P and P'.
- **You have to read all of** T ($\Omega(n)$). Suppose position $T[i]$ is never read. The adversary flips that character, potentially creating or destroying a match right around position i. Since the algorithm never looked at $T[i]$, it has no way to tell the difference.

The two bounds add up: $\Omega(m) + \Omega(n) = \Omega(n + m)$. □

This $\Omega(n+m)$ lower bound applies to any *algorithm, not just naive.*

The naive algorithm achieves $\mathcal{O}(nm)$ in the worst case, which is far from this lower bound. KMP will close the gap, achieving $\Theta(n + m)$.

Properties

We evaluate string matching algorithms along four axes:

Online. The algorithm processes T strictly left-to-right, one character at a time, and can emit results before reading future characters of T. Useful when T arrives as a stream.

Real-time. A stronger requirement: each new character of T is fully processed in $\mathcal{O}(1)$ *worst-case* time before the next one is read.

Blind (no backtracking). The algorithm never re-reads a position of T: each character of T is accessed at most once. A blind algorithm is necessarily online; the converse is false (the naive algorithm is online but not blind).

Succinct. The algorithm uses only $\mathcal{O}(1)$ extra space beyond storing P and T (no large auxiliary data structures).

Algorithm	Online	Real-time	Blind	Succinct
Naive	✓	×	×	✓
KMP	✓	×	✓	×
KMP (sp')	✓	✓	✓	×

The naive algorithm backtracks in T: at each new shift s it re-reads characters already compared at shift $s-1$.

The naive algorithm is **online** (it reads T left-to-right) but not **blind**: at each new shift it backtracks and re-compares characters of T that were already seen. It is also not **real-time**: a single character of T can trigger up to $\mathcal{O}(m)$ comparisons.

This naive approach basically starts over from scratch at each shift, without leveraging precious information from previous comparisons.

1.1.3 Knuth-Morris-Pratt (KMP) algorithm

Intuition

What if we could somehow "remember" the information from a previous comparison, and use it to skip some of the checks at the next shift? Let's introduce the intuition behind the Knuth-Morris-Pratt (KMP) algorithm, with an example.

Example 1.1.6. Let $P =$ `abcdeabdef` and suppose we are running the naive algorithm at shift $s = 3$ over some text T whose characters at positions 3–9 are `abcdeab`. We match $P[1 \dots 7]$ successfully, then hit a mismatch at $P[8] =$ `d` vs $T[10] =$ `x`.

As shown in Figure 1.3, at this point the naive algorithm would discard everything and try shift $s = 4$. But notice that the matched portion `abcdeab` starts *and* ends with $\alpha =$ `ab`: the prefix $P[1 \dots 2]$ and the suffix $P[6 \dots 7]$ of the matched region are identical. This means that when we slide P forward, the α that was at the end of the match is now perfectly aligned with the α at the start of P — we can jump directly to shift $s = 8$ and resume comparing from $P[3]$, skipping five shifts entirely.

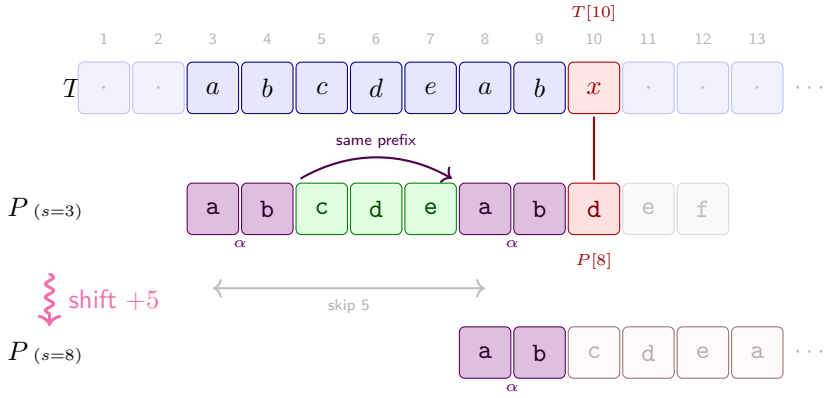
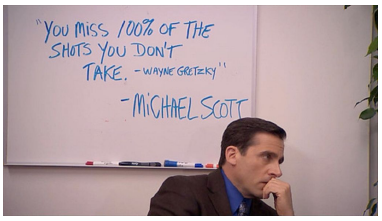


Figure 1.3: KMP shift intuition with $P =$ `abcdeabdef`. At shift $s = 3$, the algorithm matches $P[1 \dots 7] =$ `abcdeab` then hits a mismatch at $P[8]$. The matched portion ends with $\alpha =$ `ab`, which also appears at the very start of P — so instead of shifting by 1, we jump directly to $s = 8$, re-aligning α and resuming from $P[3]$.

Basically what the example shows is that if we take the longest prefix of P that is also a suffix of the **matched** portion of P, we can shift the pattern so that this prefix is now aligned with the suffix — and we can resume the search from the character right after this prefix, since we know it must match (otherwise we would have had a mismatch earlier). In other words we have saved some work (namely, five shift indexes) by leveraging the information from the previous comparisons, instead of starting over from scratch, from the very next shift.

*When we say suffix here we mean suffix of the **matched** portion of P, not of P itself. In the example, α is a suffix of $P[1 \dots 7]$ but not of the whole P.*

But is this correct? Or do we miss some potential matches by jumping directly to $s = 8$?



"You miss 100% of the shots you don't take." — Michael Scott

Well, it turns out that this is indeed correct, and we won't miss any matches

by doing this. The KMP algorithm is based on this very idea, and it guarantees that we will find all occurrences of P in T without missing any, while also achieving a much better time complexity than the naive approach.

Preprocessing: computing *sp* values

To exploit this idea, we need to precompute for every prefix $P[1 \dots i]$ its *longest proper prefix that is also a suffix*. We write this length as $\mathrm{sp}_i(P)$.

A proper *prefix/suffix excludes the string itself;* $\mathrm{sp}_i(P)$ *is thus an integer.*

Definition 1.1.7 (*sp* values)**.** Let P be a pattern of length m. For each $i \in \{1, \dots, m\}$, define

$$\mathrm{sp}_i(P) \;=\; \max\{\, k < i \mid P[1 \dots k] = P[i-k+1 \dots i] \,\},$$

i.e. the length of the longest proper prefix of $P[1 \dots i]$ that is also a suffix of $P[1 \dots i]$.

The key observation is that $\mathrm{sp}_i(P)$ can be computed recursively from $\mathrm{sp}_{i-1}(P)$, giving an efficient $\mathcal{O}(m)$ preprocessing algorithm. There are two cases:

1. **Extension is possible.** Let $k = \mathrm{sp}_{i-1}(P)$. If $P[k+1] = P[i]$, then the longest prefix-suffix of $P[1 \dots i]$ is simply one character longer than that of $P[1 \dots i-1]$, so $\mathrm{sp}_i(P) = k+1$.

   Example: with $P =$ `abcdeabdef` at $i = 7$, we have $\mathrm{sp}_6(P) = 1$ (the prefix-suffix of `abcdea` is `a`, length 1). We check $P[2] =$ `b` against $P[7] =$ `b`: they match, so $\mathrm{sp}_7(P) = 2$ (see Figure 1.4).

2. **Extension is not possible.** If $P[k+1] \neq P[i]$, we cannot extend the current prefix-suffix. We then ask: what is the longest prefix-suffix of the prefix-suffix itself? In other words, we fall back to $k \leftarrow \mathrm{sp}_k(P)$ and try to extend that instead. We repeat this process until either a match is found or we reach $k = 0$, in which case $\mathrm{sp}_i(P) = 0$.

   Example: still with $P =$ `abcdeabdef` at $i = 8$, we have $\mathrm{sp}_7(P) = 2$ (the prefix-suffix of `abcdeab` is `ab`). We check $P[3] =$ `c` against $P[8] =$ `d`: no match. We fall back to $k \leftarrow \mathrm{sp}_2(P) = 0$, so we check $P[1] =$ `a` against $P[8] =$ `d`, still no match. We have reached $k = 0$, so $\mathrm{sp}_8(P) = 0$.

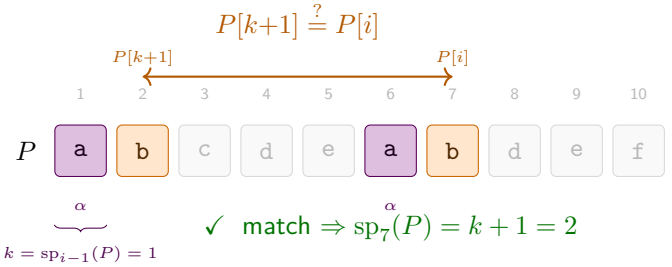


Figure 1.4: Computing $\mathrm{sp}_7(P)$ for $P =$ `abcdeabdef`. We know $k = \mathrm{sp}_6(P) = 1$: the suffix of $P[1 \dots 6]$ matching a prefix of P has length 1 (namely $\alpha =$ `a`, violet). We compare $P[k+1] = P[2] =$ `b` against $P[i] = P[7] =$ `b` (orange): they match, so $\mathrm{sp}_7(P) = 2$.

The resulting algorithm is the following:

Algorithm 2: Computing sp values

Input: Pattern $P[1 \dots m]$
Output: Array $sp[1 \dots m]$ where $sp[i] = \mathrm{sp}_i(P)$

```
1 sp[1] ← 0
2 k ← 0            // length of current candidate prefix-suffix
3 for i ← 2 to m do
4 |   while k > 0 and P[k+1] ≠ P[i] do
5 |   |   k ← sp[k]                                  // fall back
6 |   if P[k+1] = P[i] then
7 |   |   k ← k+1
8 |   sp[i] ← k
9 return sp
```

Remark 1.1.8 (Why fall back to $sp[k]$?). At every point in the loop, k has a precise meaning: it is the length of the longest prefix of P that currently matches a suffix of $P[1 \dots i{-}1]$. In other words,

$$P[1 \dots k] \;=\; P[i-k \dots i-1],$$

the k characters immediately before position i are exactly the first k characters of the pattern. When $P[k+1] = P[i]$ we can extend this match by one: $sp[i] = k+1$. When $P[k+1] \neq P[i]$ the match of length k cannot be extended, and we need a shorter candidate $j < k$ that still satisfies the same property: $P[1 \dots j] = P[i-j \dots i-1]$.

Key insight. $P[1 \dots k]$ is not an arbitrary string — it is a *prefix* of P. Any valid candidate j must be a prefix of P that also appears as a suffix of $P[1 \dots i{-}1]$. Two simple observations handle this:

- *Prefix of a prefix is a prefix*: if $P[1 \dots j]$ is a prefix of $P[1 \dots k]$, it is a prefix of P.
- *Suffix of a suffix is a suffix*: since $P[1 \dots k]$ is a suffix of $P[1 \dots i{-}1]$ (by the invariant), any suffix of $P[1 \dots k]$ of length j is also a suffix of $P[1 \dots i{-}1]$.

Therefore j is a valid candidate *if and only if* $P[1 \dots j]$ is a *border* of $P[1 \dots k]$ (a string that is both a prefix and a suffix of it). The longest such border is $sp[k]$, already computed during preprocessing. Any value strictly between $sp[k]$ and k is, by definition of $sp[k]$, not a border of $P[1 \dots k]$ and therefore not a valid candidate — no check is needed. We jump directly to $k \leftarrow sp[k]$ and repeat. Figure 1.5 shows this chain for $i = 6$, $k = 3$.

Note 1.1.9 (A fallback that succeeds). Let $P =$ `ababcababa` and consider $i = 10$. We have $k = 4$ since $P[1 \dots 4] = P[6 \dots 9] =$ `abab`. Check $P[5] =$ `c` vs. $P[10] =$ `a`: mismatch. Could $k' = 3$ be a valid candidate? It would need $P[1 \dots 3]$ to be a border of `abab`, i.e. `aba` = `bab`: no. The longest border of `abab` is `ab` (length 2), so $sp[4] = 2$. We fall back to $k = 2$: $P[1 \dots 2] = P[8 \dots 9] =$ `ab`. Check $P[3] =$ `a` vs. $P[10] =$ `a`: **match**! Hence $sp[10] = 3$; skipping $k' = 3$ was the only correct move.

FORMAL DETAILS — **Why no candidate between $sp[k]$ and k exists**

Suppose for contradiction that $sp[k] < k' < k$ and that k' satisfies the same property as k, i.e. $P[1 \dots k'] = P[i-k' \dots i-1]$. Since $k' < k$, the string $P[i-k' \dots i-1]$ is a suffix of $P[i-k \dots i-1] = P[1 \dots k]$. Combined with $P[1 \dots k']$ being a prefix of P, this makes $P[1 \dots k']$ a border of $P[1 \dots k]$ of length $k' > sp[k]$, contradicting $sp[k]$ being the *longest* proper border of $P[1 \dots k]$. □

FORMAL DETAILS — **Correctness of the sp preprocessing algorithm**

We prove that at the end of iteration i, $sp[i]$ equals the length of the longest proper border of $P[1 \dots i]$.

Precondition	$P[1 \dots m]$ is given; $sp[1] = 0$ and $k = 0$ are set before the loop.
Loop guard	i ranges from 2 to m.
Invariant	At the *start* of iteration i: $k = sp[i-1]$, i.e. k is the length of the longest proper border of $P[1 \dots i-1]$ (and in particular $P[1 \dots k] = P[i-k \dots i-1]$).
Postcondition	$sp[i]$ is correctly set for all $1 \le i \le m$.
Base case	Before the first iteration ($i = 2$), $k = 0 = sp[1]$ since $P[1 \dots 1]$ has no proper border. ✓
Preservation	Assume the invariant holds at the start of iteration i, i.e. $k = sp[i-1]$. The inner **while** loop traverses the border chain $k \to sp[k] \to sp[sp[k]] \to \cdots$, stopping as soon as $P[k+1] = P[i]$ or $k = 0$. By the border characterisation (a candidate j is valid iff $P[1 \dots j]$ is a border of $P[1 \dots k]$), this finds the longest extendable border. The subsequent **if** either increments k by one or leaves $k = 0$. In both cases $sp[i] \leftarrow k$ is correct, and $k = sp[i]$ is the invariant for iteration $i+1$. ✓
Postcondition	When the loop exits ($i = m + 1$), all $sp[i]$ have been correctly computed. ✓

Step-by-step: Computing sp values

Let's trace the algorithm on $P =$ `ababaca`.

1. $i = 1$**:** By definition $sp[1] = 0$. $k \leftarrow 0$.
2. $i = 2$**:** $k = 0$, $P[1] =$ `a` $\neq P[2] =$ `b`. $sp[2] = 0$.
3. $i = 3$**:** $k = 0$, $P[1] =$ `a` $= P[3] =$ `a`. $k \leftarrow k + 1 = 1$. $sp[3] = 1$.
4. $i = 4$**:** $k = 1$, $P[2] =$ `b` $= P[4] =$ `b`. $k \leftarrow k + 1 = 2$. $sp[4] = 2$.
5. $i = 5$**:** $k = 2$, $P[3] =$ `a` $= P[5] =$ `a`. $k \leftarrow k + 1 = 3$. $sp[5] = 3$.
6. $i = 6$**:** $k = 3$. The invariant gives $P[1 \dots 3] = P[3 \dots 5] =$ `aba`. We check $P[4] =$ `b` against $P[6] =$ `c`: mismatch, so we fall back. The fallback chain is shown in Figure 1.5:
   - $k \leftarrow sp[3] = 1$: the longest border of `aba` is `a` (length 1). Check $P[2] =$ `b` against $P[6] =$ `c`: mismatch.

- $k \leftarrow sp[1] = 0$: the longest border of a is empty. Check $P[1] =$ a against $P[6] =$ c: mismatch.
- $k = 0$: no candidate left. $sp[6] = 0$.

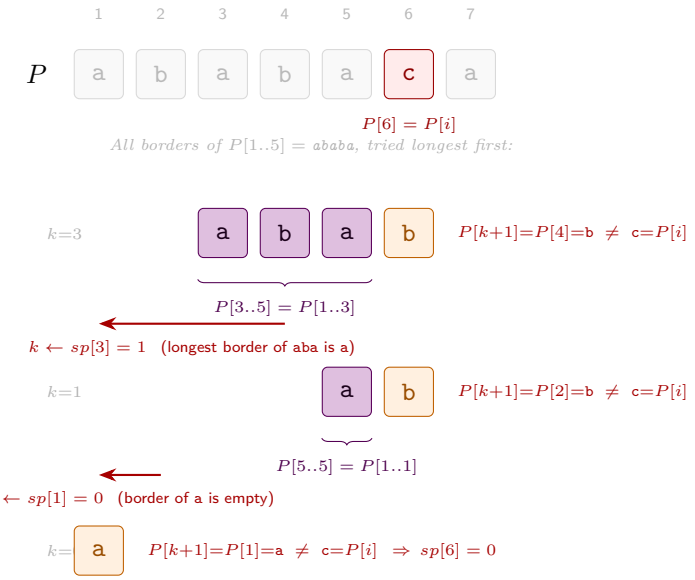


Figure 1.5: Fallback chain at step $i = 6$ for $P =$ ababaca. The three rows show *all* borders of $P[1..5] =$ ababa (lengths 3, 1, 0), tried longest first. For each candidate k, the violet cells show the matched suffix $P[i-k..i-1] = P[1..k]$; the orange cell is the character $P[k+1]$ we would need at position i to extend it. All three fail, so $sp[6] = 0$.

7. $i = 7$: $k = 0$, $P[1] =$ a $= P[7] =$ a. $k \leftarrow k + 1 = 1$. $sp[7] = 1$.

i	1	2	3	4	5	6	7
$P[i]$	a	b	a	b	a	c	a
$sp[i]$	0	0	1	2	3	0	1

> *Remark* 1.1.10. The "fall back to the sp value of the current sp value" is the recursive heart of KMP: the same shift logic used during the search phase is also used during preprocessing.

> *Exercise* 1.1.11. Design an algorithm that, given a pattern P of length m, computes $\mathrm{sp}_i(P)$ for every $i \in \{1, \ldots, m\}$ in $\mathcal{O}(m)$ time.
>
> *Hints:*
>
> 1. Try to express $\mathrm{sp}_i(P)$ in terms of $\mathrm{sp}_{i-1}(P)$. What is the relationship between the longest prefix-suffix of $P[1 \ldots i]$ and that of $P[1 \ldots i-1]$?
> 2. There are two cases depending on whether $P[\mathrm{sp}_{i-1}(P) + 1]$ equals $P[i]$ or not. What can you do in each case?
> 3. For the harder case, think recursively: if you cannot extend a prefix-suffix of length k, which shorter candidate should you try next?

Exercise 1.1.12 (Period of a string). The *period* of a string S of length n is the smallest integer $p \geq 1$ such that $S[i] = S[i+p]$ for all $1 \leq i \leq n-p$. Equivalently, p is the length of the shortest string Π such that S is a prefix of $\Pi^\infty = \Pi\Pi\Pi\Pi\cdots$

Design an $\mathcal{O}(n)$-time algorithm that, given S, computes its period.

Hints:

1. Compute the *sp* values of S.
2. What is the relationship between $sp_n(S)$ and the period of S? In particular, is $n - sp_n(S)$ always a period?
3. Is it always the *shortest* period? What additional condition on $sp_n(S)$ guarantees minimality?

INTERMEZZO — **Amortised analysis — the credit method**

Some algorithms contain operations that look expensive in isolation but are cheap *on average* over any sequence of calls. *Amortised analysis* makes this precise by distributing the cost of occasional expensive operations across the many cheap ones that made them possible.

A clean way to formalise this is the **credit method**.

Definition 1.1.13 (Credit method)**.** Assign each operation an *amortised cost* $\hat{c}$ (which we choose freely). When an operation with real cost c runs:

- if $\hat{c} > c$, the surplus $\hat{c} - c$ is deposited as *credits* into a bank;
- if $\hat{c} < c$, the deficit $c - \hat{c}$ is withdrawn from the bank.

Invariant: the bank balance must never go negative.

Consequence: since the bank starts at 0 and never goes below 0,

$$\sum_i c_i \;\leq\; \sum_i \hat{c}_i,$$

i.e. the total real cost is bounded by the total amortised cost.

Example: stack with push and multi-pop. Consider a stack supporting two operations:

- PUSH(x): push element x onto the stack; real cost 1.
- MULTI-POP(k): pop the top $\min(k, |S|)$ elements; real cost = number of elements actually popped.

A single MULTI-POP can cost $\Theta(n)$, so a naive worst-case analysis gives $\mathcal{O}(n^2)$ for n operations. Amortised analysis does much better.

Credit assignment. Attach 1 credit to each element at the moment it is pushed:

- PUSH: amortised cost $\hat{c} = 2$ (pay 1 for the real push, deposit 1 credit *on the element*).
- MULTI-POP(k): amortised cost $\hat{c} = 0$ (use the 1 credit stored on each popped element to pay for its removal).

The bank balance equals the number of elements on the stack, which is always ≥ 0, so the invariant holds.

Example 1.1.14 (Credit trace for a stack)**.** The table below traces the sequence PUSH(a), PUSH(b), PUSH(c), MULTI-POP(2), PUSH(d), PUSH(e), MULTI-POP(3).

Operation	Stack after	Real cost	Amortised cost	Bank
PUSH(a)	$[a]$	1	2	1
PUSH(b)	$[a, b]$	1	2	2
PUSH(c)	$[a, b, c]$	1	2	3
MULTI-POP(2)	$[a]$	2	0	1
PUSH(d)	$[a, d]$	1	2	2
PUSH(e)	$[a, d, e]$	1	2	3
MULTI-POP(3)	$[\,]$	3	0	0
Total		**10**	**10**	

The bank never goes negative, confirming the invariant. The total real cost 10 is indeed $\leq$ the total amortised cost 10.

Lemma 1.1.15. *Any sequence of n PUSH and MULTI-POP operations on an initially empty stack costs $\mathcal{O}(n)$ in total.*

Proof. There are at most n pushes, each with amortised cost 2, and all MULTI-POPs have amortised cost 0. The total amortised cost is therefore at most $2n$. By the credit-method bound, the total real cost is at most $2n = \mathcal{O}(n)$. □

Application to KMP preprocessing

The sp-computation algorithm runs in $\mathcal{O}(m)$ total time despite the nested loops. We apply the credit method with k as the potential: credits measure how much k can still decrease.

- k increases by at most 1 per outer iteration (there are $m - 1$ iterations, so k grows at most $m - 1$ times in total).
- Each execution of the inner **while** body sets $k \leftarrow sp[k] < k$, strictly decreasing k.
- $k \geq 0$ always holds.

Assign amortised cost 2 to each outer step that increments k (real cost 1 plus deposit 1 credit), and amortised cost 0 to each inner iteration that decrements k (use the saved credit). The bank balance equals $k \geq 0$, so the invariant holds, and the total number of inner iterations is at most the total number of increments, which is $\mathcal{O}(m)$. Together with the $\mathcal{O}(m)$ overhead of the outer loop, the total preprocessing time is $\mathcal{O}(m)$.

Example 1.1.16. Consider the pattern $P =$ `aaaaa` of length 5. The sp values are $sp[1] = 0$, $sp[2] = 1$, $sp[3] = 2$, $sp[4] = 3$, $sp[5] = 4$. The variable

k starts at 0 and increases to 1, then to 2, then to 3, then to 4, as we process each character of P. There are no decreases in this case, so the inner loop runs only once per outer iteration, for a total of $\mathcal{O}(m)$ time.

The search phase

Once the sp array is computed, we scan T left-to-right, maintaining j: the number of characters of P currently matched.

Algorithm 3: KMP search

Input: Text $T[1 \ldots n]$, pattern $P[1 \ldots m]$, precomputed $sp[1 \ldots m]$
Output: All starting positions s where P occurs in T

```
1 j ← 0                                   // characters of P matched so far
2 for i ← 1 to n do
3     while j > 0 and P[j + 1] ≠ T[i] do
4         j ← sp[j]                       // fall back along the border chain
5     if P[j + 1] = T[i] then
6         j ← j + 1
7     if j = m then
8         output i − m + 1                // occurrence starting here
9         j ← sp[j]                       // continue searching
```

Figure 1.6 traces the search on a concrete example, showing how sp values let the algorithm carry previously matched characters across fallbacks instead of starting from scratch.

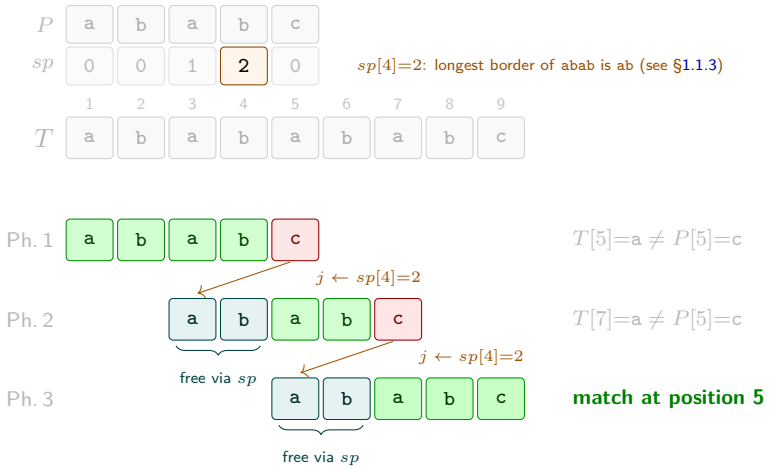


Figure 1.6: KMP search for P=ababc (m=5) in T=abababac (n=9). The sp array for P is shown at the top (cf. the preprocessing in §1.1.3). Green: characters newly matched in this phase. Teal: characters carried for free from the previous phase via $j \leftarrow sp[j]$. Red: mismatch. After each fallback, the algorithm reuses the last $sp[j]$ matched characters without any comparison, then continues from where it left off in T.

The same amortised argument applies: j increases by at most 1 per outer step and each inner fallback strictly decreases $j \geq 0$, so the inner loop runs $\mathcal{O}(n)$ times in total.

Theorem 1.1.17. *The KMP algorithm finds all occurrences of P in T in $\mathcal{O}(n+m)$ time.*

1.1.4 Z-Algorithm

Definition

The Z-algorithm is a second linear-time preprocessing approach. Where $\mathrm{sp}_i(P)$ looks *backward* (longest prefix that is also a suffix of $P[1\dots i]$), the Z-value Z_i looks *forward* from position i.

Definition 1.1.18 (Z values)**.** Let A be a string of length n. For $i \geq 2$, define

$$Z_i(A) \;=\; \begin{cases} 0 & \text{if } A[i] \neq A[1], \\ \max\{\, \ell > 0 \mid A[1,\ell] = A[i,\, i+\ell-1] \,\} & \text{otherwise.} \end{cases}$$

In words, $Z_i(A)$ is the length of the longest prefix of A that matches A starting at position i, as shown in Figure 1.7.

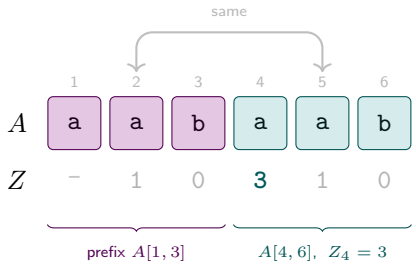


Figure 1.7: Z values for $A = \texttt{aabaab}$ ($n = 6$). The prefix $A[1,3] = \texttt{aab}$ (violet) reappears at position 4 (teal), giving $Z_4 = 3$. Also $Z_2 = Z_5 = 1$ since $A[1] = A[2] = A[5] = \texttt{a}$, and $Z_3 = Z_6 = 0$ since $A[3] = A[6] = \texttt{b} \neq A[1]$. Z_1 is left undefined.

Connection to pattern matching. Choose a separator $\$ \notin \Sigma$ and form $A = P\$T$ (of length $m + 1 + n$). For any index i in the T-part of A (i.e. $i > m+1$):

$$Z_i(P\$T) = m \quad \Longleftrightarrow \quad P \text{ occurs in } T \text{ starting at position } i - m - 1.$$

The separator prevents Z_i from exceeding m, so equality with m detects all occurrences.

FORMAL DETAILS — **Proof of the connection**

Write $A = P\$T$ and fix $i > m+1$ (i.e. i is in the T-part of A).

The separator bounds $Z_i \leq m$. If $Z_i > m$ then $A[i\dots i+m] = A[1\dots m+1]$, so in particular $A[i+m] = A[m+1] = \$$. But $i + m > 2m + 1 > m + 1$, so $A[i+m]$ lies in the T-part of A, hence $A[i+m] \in \Sigma$. Since $\$ \notin \Sigma$ we have a contradiction: $Z_i \leq m$.

($\Rightarrow$) Suppose $Z_i = m$. Then $A[i\dots i+m-1] = A[1\dots m] = P$. Since $A[i+j-1] = T[i-m-1+j-1]$ for $j = 1, \dots, m$, this gives

$T[i-m-1\ldots i-1] = P$, i.e. P occurs at position $i-m-1$ in T.

($\Leftarrow$) Suppose P occurs at position $j = i-m-1$ in T, i.e. $T[j\ldots j+m-1] = P$. Then $A[i\ldots i+m-1] = T[j\ldots j+m-1] = P = A[1\ldots m]$, so $Z_i \geq m$. Combined with $Z_i \leq m$ we get $Z_i = m$. □

Example 1.1.19. Let $P = \texttt{aba}$ ($m = 3$) and $T = \texttt{ababac}$ ($n = 6$). Form $A = \texttt{aba\$ababac}$ and compute Z values for positions $i > 4$:

i	1	2	3	4	5	6	7	8	9	10
$A[i]$	a	b	a	$	a	b	a	b	a	c
Z_i	–	0	1	0	**3**	0	**3**	0	1	0

The two positions with $Z_i = 3 = m$ are $i = 5$ and $i = 7$, corresponding to occurrences of P in T starting at positions $5-3-1 = 1$ and $7-3-1 = 3$. Indeed $T[1\ldots 3] = \texttt{aba}$ and $T[3\ldots 5] = \texttt{aba}$. ✓

Computing Z values: the Z-box algorithm

A naive computation of all Z values costs $\mathcal{O}(n^2)$. The Z-algorithm achieves $\mathcal{O}(n)$ by maintaining a *Z-box* $[l, r]$: the interval with maximum right endpoint r seen so far such that $A[l, r] = A[1, r-l+1]$, i.e.

$$r = l + Z_l(A) - 1 \quad \text{(maximised over all processed positions).}$$

Figure 1.8 shows this setup schematically.

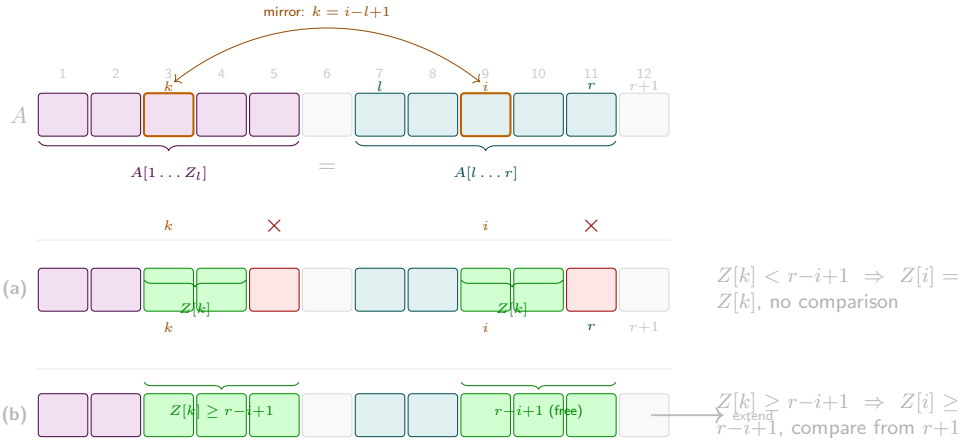


Figure 1.8: Z-box schematic (positions l=7, r=11, k=3, i=9). Violet: the prefix $A[1\ldots Z_l]$ that established the box. Teal: the Z-box $A[l\ldots r]$, a copy of that prefix. Orange border: mirror pair (k, i) with $k = i-l+1$. Green: the $Z[k]$ characters that match at both k and i (free, by the mirror property). Red ×: the mismatch that terminated $Z[k]$, reflected at i. Case (a): the mismatch lies inside $[l, r]$, so $Z[i] = Z[k]$ at no cost. Case (b): the match reaches r, so we must compare from $r+1$. See Figure 1.9 for a concrete step-by-step trace.

The following two cases (Figure 1.10 and Figure 1.9) distinguish how we compute Z_i:

1. $i > r$ **(outside the box).** No cached information is available. Compare $A[1], A[2], \ldots$ against $A[i], A[i+1], \ldots$ directly until a mismatch, setting Z_i to the number of matches. If $Z_i > 0$, update $l \leftarrow i$ and $r \leftarrow i + Z_i - 1$.

2. $i \leq r$ **(inside the box).** Let $k = i - l + 1$ be the *mirror* of i in the prefix: $A[i, r] = A[k,\, r - l + 1]$ and Z_k is already known.
   (a) $Z_k < r - i + 1$**:** the Z-box at k is strictly shorter than the remaining portion of the current box. The match at i cannot extend further: $Z_i = Z_k$, no comparisons needed.
   (b) $Z_k \geq r - i + 1$**:** the cached region is exhausted. Start comparing from $A[r + 1]$ against $A[r - i + 2]$ until a mismatch, then update $l \leftarrow i$ and $r \leftarrow i + Z_i - 1$.

Figure 1.9 traces the full execution on $A =$ `aabaaab`, showing how $[l, r]$ evolves across all three cases.

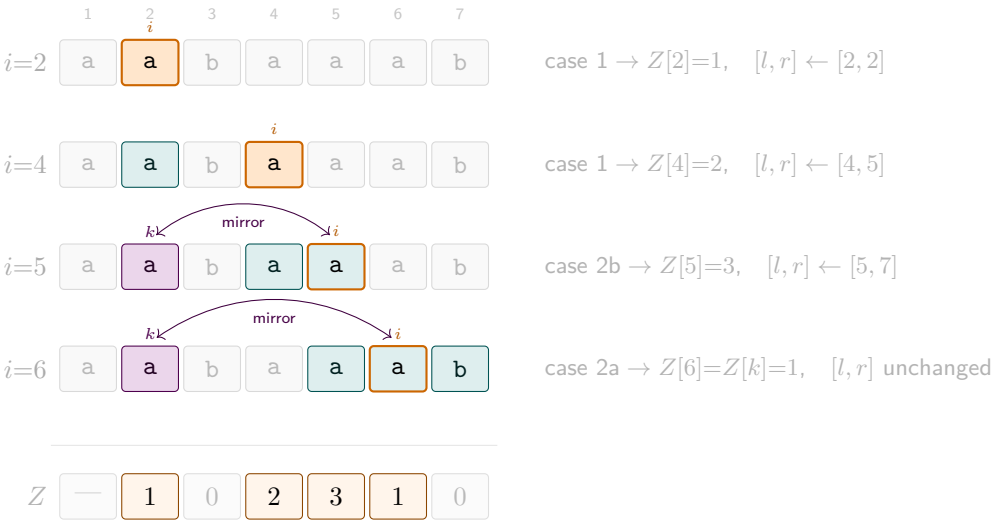
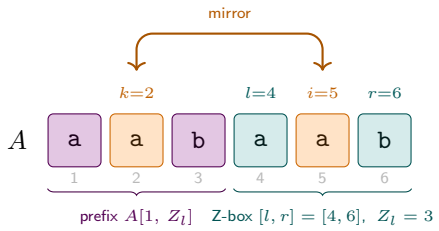


Figure 1.9: Z-box algorithm on $A =$ `aabaaab` (steps i=3 and i=7 omitted: Z=0, no update). Teal: current Z-box $[l, r]$. Orange border: current position i (filled teal when i is inside the box). Violet: mirror $k = i{-}l{+}1$, with arc showing the mirror correspondence. Bottom row: final Z array (non-zero entries highlighted).



$$Z_k = Z_2 = 1 \;<\; r{-}i{+}1 = 2 \;\Rightarrow\; Z_5 = Z_k = 1 \quad \text{(no comparisons, case (a))}$$

Figure 1.10: Z-box algorithm on $A =$ `aabaab`, step $i = 5$. The Z-box $[l, r] = [4, 6]$ (teal) records that $A[4, 6] = A[1, 3] =$ `aab`. The mirror of $i = 5$ in the prefix is $k = i{-}l{+}1 = 2$ (orange). Since $Z_k = Z_2 = 1$ is strictly less than the remaining box width $r{-}i{+}1 = 2$, the match at i cannot extend beyond Z_k — we set $Z_5 = 1$ immediately without any character comparisons (case (a)).

Algorithm 4: Z-Algorithm

Input: String $A[1 \dots n]$
Output: Array $Z[2 \dots n]$ with $Z[i] = Z_i(A)$

```
1  l ← 0
2  r ← 0
3  for i ← 2 to n do
4      if i > r then
5          Z[i] ← 0
6          while i + Z[i] ≤ n and A[1 + Z[i]] = A[i + Z[i]] do
7              Z[i] ← Z[i] + 1
8      else
9          k ← i − l + 1
10         if Z[k] < r − i + 1 then
11             Z[i] ← Z[k]          // fully inside - no comparisons
12         else
13             Z[i] ← r − i + 1
14             while i + Z[i] ≤ n and A[1 + Z[i]] = A[i + Z[i]] do
15                 Z[i] ← Z[i] + 1
16     if Z[i] > 0 and i + Z[i] − 1 > r then
17         l ← i
18         r ← i + Z[i] − 1
19 return Z
```

FORMAL DETAILS — **Correctness of the Z-box algorithm**

We prove that the algorithm sets $Z[i] = Z_i(A)$ for every $i \geq 2$.

Precondition	$A[1 \dots n]$ is given; $l = r = 0$ before the loop.
Loop guard	i ranges from 2 to n.
Invariant	At the *start* of iteration i: (i) $Z[j]$ is correctly set for all $2 \leq j < i$, and (ii) $[l, r]$ is the Z-box with the largest right endpoint seen so far, i.e. $A[l \dots r] = A[1 \dots r{-}l{+}1]$ (with $l = r = 0$ meaning no box yet).
Postcondition	$Z[i] = Z_i(A)$ for all $i \in \{2, \dots, n\}$.
Base case	Before $i = 2$: Z is empty and $l = r = 0$, so the invariant holds vacuously. ✓
Preservation	Assume the invariant holds at the start of iteration i. We analyse the two cases. *Case 1:* $i > r$. The while loop compares $A[1], A[2], \dots$ against $A[i], A[i{+}1], \dots$ directly and stops at the first mismatch, so $Z[i]$ is set to the correct value by definition. If $Z[i] > 0$, the box is updated to $[l, r] = [i,\ i{+}Z[i]{-}1]$, which has a larger r than before. ✓ *Case 2a:* $i \leq r$ *and* $Z[k] < r{-}i{+}1$ *(mirror strictly inside the box).* Let $k = i{-}l{+}1$. From the invariant, $A[l \dots r] = A[1 \dots r{-}l{+}1]$, so $A[i{+}j] = A[k{+}j]$

	for $j = 0, \dots, r-i$. In particular $A[i \dots i+Z[k]-1] = A[k \dots k+Z[k]-1] = A[1 \dots Z[k]]$. Since $Z[k] < r-i+1$, position $k+Z[k]$ still lies inside $[1, r-l+1]$, so $A[i+Z[k]] = A[k+Z[k]] \neq A[1+Z[k]]$ (the character that ended the match at k). Therefore $Z[i] = Z[k]$. The box $[l, r]$ is not updated since $i+Z[i]-1 < r$. ✓ *Case 2b: $i \leq r$ and $Z[k] \geq r-i+1$ (mirror reaches or exceeds the box boundary).* The same translation gives $A[i \dots r] = A[1 \dots r-i+1]$, so $Z[i] \geq r-i+1$ is guaranteed. The algorithm initialises $Z[i] = r-i+1$ and extends it by comparing $A[r+1], A[r+2], \dots$ directly until a mismatch, exactly as in Case 1. The box is then updated to $[i, i+Z[i]-1]$. ✓
Postcondition	When the loop exits, all $Z[i]$ have been correctly computed by the case analysis above. ✓

Amortised $\mathcal{O}(n)$ complexity. The variable r is non-decreasing and bounded by n, so it increments at most n times. Every comparison in the **while** loops either advances r (paid by the budget) or ends the step without advancing r (at most one per outer step i). Hence the total number of comparisons is $\mathcal{O}(n)$.

From Z values to *sp* values

The *sp* values of P can be read off directly from its Z-array.

$Z_j(P) = k$ means $P[j \dots j+k-1] = P[1 \dots k]$, so $P[1 \dots k]$ is a prefix-suffix of $P[1 \dots i]$ for $i = j + k - 1$. We say j *maps to* i when $i = j + Z_j(P) - 1$.

Proposition 1.1.20. *For each $i \in \{2, \dots, m\}$,*

$$\mathrm{sp}_i(P) \;=\; Z_{j^*}(P), \qquad \textit{where} \quad j^* = \min\{\, j \geq 2 \mid j + Z_j(P) - 1 = i \,\}.$$

Proof. Any j mapping to i gives a prefix-suffix of $P[1 \dots i]$ of length $Z_j(P) = i - j + 1$. The minimum such j maximises $i - j + 1$, yielding the longest prefix-suffix, which is $\mathrm{sp}_i(P)$ by definition. □

Exercise 1.1.21. Using the Z-array of P, design an $\mathcal{O}(m)$ algorithm that computes $\mathrm{sp}_i(P)$ for every $i \in \{1, \dots, m\}$.

1.1.5 Strong borders and real-time KMP

The real-time bottleneck

KMP with plain *sp* values is *blind* (it never re-reads T) but not *real-time*: a single new character of T can trigger a fallback chain costing $\mathcal{O}(m)$ in the worst case. The fix is to collapse the entire chain into a single precomputed lookup.

Strong border (*sp′*)

Definition 1.1.22 (Strong border)**.** For $i \in \{1, \ldots, m-1\}$, define

$$\mathrm{sp}_i(P)' \;=\; \max\{\, k \le \mathrm{sp}_i(P) \mid k = 0 \text{ or } P[k+1] \neq P[i+1] \,\}.$$

This is the length of the longest proper prefix-suffix of $P[1 \ldots i]$ whose next character differs from $P[i+1]$.

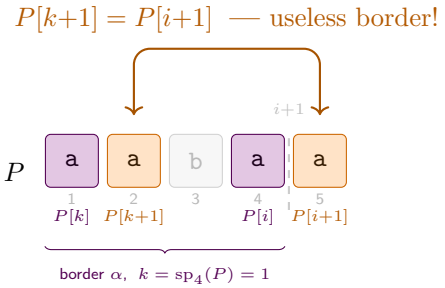


Figure 1.11: Strong border for $P = \texttt{aabaa}$ at $i = 4$. The border $\alpha = \texttt{a}$ (violet, $k = \mathrm{sp}_4(P) = 1$) satisfies $P[1] = P[4]$, but its "next character" $P[k{+}1] = P[2] = \texttt{a}$ (orange) equals $P[i{+}1] = P[5] = \texttt{a}$. Any mismatch on $P[5]$ with text character $x \neq \texttt{a}$ would *also* fail immediately at $P[k{+}1]$ after the fallback — the border is useless. Hence $\mathrm{sp}_4(P)' = 0$: skip straight to the start.

Any mismatch on $P[i+1]$ with a text character x would also fail at the next border $k = \mathrm{sp}_i(P)$ if $P[k+1] = P[i+1]$ — we say such borders are *useless* (see Figure 1.11). $\mathrm{sp}_i(P)'$ skips all of them in one step, guaranteeing $\mathcal{O}(1)$ worst-case time per character of T.

Character-indexed borders and the DFA

For a complete real-time solution we precompute, for each position i and character $x \in \Sigma$, the longest border of $P[1 \ldots i]$ whose next character is exactly x:

Definition 1.1.23 ($sp_{i,x}$ values)**.** For $i \in \{0, 1, \ldots, m\}$ and $x \in \Sigma$,

$$\mathrm{sp}_{i,x}(P) \;=\; \max\{\, k \le \mathrm{sp}_i(P) \mid P[k+1] = x \,\},$$

with default 0 when no such k exists.

The table $\{\mathrm{sp}_{i,x}(P)\}_{i,x}$ is precisely the *transition function* of the deterministic finite automaton (DFA) for pattern P: state i encodes "i characters of P matched so far", and on reading x the automaton transitions to $\mathrm{sp}_{i,x}(P) + \mathbf{1}[P[i+1] = x]$. With this table, each character of T is processed in $\mathcal{O}(1)$ worst-case time, making KMP truly **real-time**.

The DFA has $m+1$ states and $|\Sigma|$ transitions per state, so the table takes $\mathcal{O}(m|\Sigma|)$ space and time to build.

Online vs. real-time: KMP vs. Z-algorithm. The Z-algorithm requires the *entire* string before it can report results — the Z-box $[l, r]$ may depend on characters far ahead — so it is *not online*. KMP with sp' processes T one character at a time in $\mathcal{O}(1)$ worst-case per character, making it both **online** and **real-time** — the key practical advantage over the Z-algorithm.

KMP with sp' (or the full $sp_{i,x}$ table) is also blind *and processes each new character in $\mathcal{O}(1)$ worst-case, which is why it appears as real-time in the comparison table above.*

1.1.6 A Comprehensive Trace: KMP and Z-Algorithm

To solidify these concepts, let's trace both algorithms on a concrete example.

KMP Step-by-Step

Consider the pattern $P =$ `ababa` and the text $T =$ `ababcababa`.

1. Preprocessing P. We compute the *sp* array for P.

i	1	2	3	4	5
$P[i]$	a	b	a	b	a
$\text{sp}_i(P)$	0	0	1	2	3

For $i = 4$, $P[1..4] =$ `abab`. Proper prefix-suffixes are `ab` (len 2) and `a` (len 1). Max is 2.

2. Searching T. We maintain a pointer i into T and a pointer j representing the length of the current match in P. The complete trace is shown in Figure 1.12.

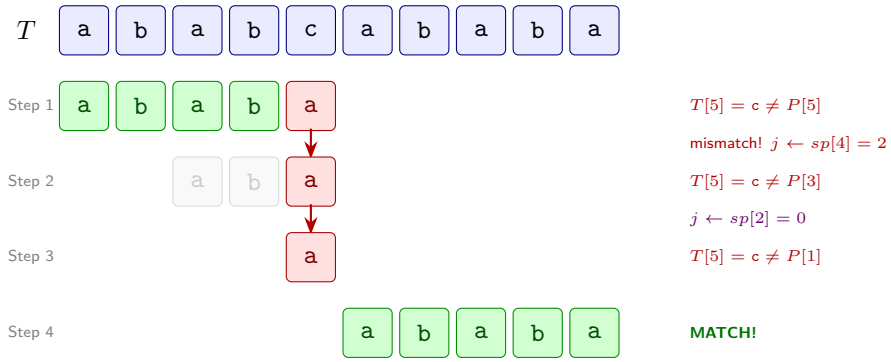


Figure 1.12: KMP trace for $P =$ `ababa` in $T =$ `ababcababa`. Step 1 finds a mismatch at $T[5]$. KMP falls back twice ($j = 4 \to 2 \to 0$) without re-reading T. Finally, Step 4 finds a complete match starting at $T[6]$.

Z-Algorithm Trace

Let's compute the Z-values for the string $S =$ `aababca`.

1. $i = 2$**:** $S[2] =$ `a` $= S[1]$. Check $S[3] =$ `b` $\neq S[2]$. $Z_2 = 1$. Box $[l, r] = [2, 2]$.
2. $i = 3$**:** $i > r = 2$. $S[3] =$ `b` $\neq S[1]$. $Z_3 = 0$. Box stays $[2, 2]$.
3. $i = 4$**:** $i > r = 2$. $S[4] =$ `a` $= S[1]$. Check $S[5] =$ `b` $= S[2]$, $S[6] =$ `c` $\neq S[3]$. $Z_4 = 2$. New box $[l, r] = [4, 5]$.
4. $i = 5$**:** $i \leq r = 5$. Mirror $k = i - l + 1 = 5 - 4 + 1 = 2$. $Z_k = Z_2 = 1$. Remaining box width $r - i + 1 = 5 - 5 + 1 = 1$. Since $Z_k \not< r - i + 1$ (Case 2b), we try to extend from $r + 1 = 6$. $S[6] =$ `c` $\neq S[3]$. No extension. $Z_5 = 1 + 0 = 1$. Box stays $[4, 5]$.
5. $i = 6$**:** $i > r = 5$. $S[6] =$ `c` $\neq S[1]$. $Z_6 = 0$.
6. $i = 7$**:** $i > r = 5$. $S[7] =$ `a` $= S[1]$. No more chars. $Z_7 = 1$.

i	1	2	3	4	5	6	7
$S[i]$	a	a	b	a	b	c	a
Z_i	–	1	0	2	1	0	1

Chapter 2

Randomized Algorithms

Chapter 3

Data Compression